WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

# Face à KAPE et EZ
# WAC (Windows Artefacts Collector)

## Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

# WAC — Windows Artefacts Collector

Face à KAPE et aux outils d'Eric Zimmerman

| | |
|---|---|
| Version du logiciel | 2.0.0 |
| Édition du document | octobre 2026 |
| Éditeur | Centre d'Excellence Cyberdéfense Aérospatiale (CEC) |
| | École de l'air et de l'espace, base aérienne 701, Salon-de-Provence |

Ce document confronte WAC aux outils de référence de la communauté : KAPE pour la collecte, les outils en ligne de commande d'Eric Zimmerman pour le décodage. Sur les mêmes pièces d'une même machine, il compare les temps d'exécution, ce que chacun prélève, la justesse des valeurs — chaque écart étant tranché par une référence indépendante —, les formats de sortie, et les traces que chacun laisse sur la machine examinée. Il dit ce que la confrontation a appris à WAC, et ce qu'elle lui reproche encore.

# Table des matières


**1 La démarche** **2**

1.1 Ce qui est comparé . . . 2

1.2 Qui a raison quand ils diffèrent . . . 2

1.3 Le banc d'essai . . . 3

**2 Synthèse** **4**

**3 Temps d'exécution** **6**

3.1 Décodage . . . 6

3.2 Collecte . . . 6

3.3 Collecte depuis un même état . . . 7

**4 Ce que chacun prélève** **9**

4.1 Les mêmes pièces, des fichiers différents . . . 9

4.2 Ce qui est consigné de chaque pièce . . . 9

**5 Traces laissées sur la machine examinée** **11**

5.1 La mesure . . . 11

5.2 Ce que chacun laisse . . . 11

5.3 Ce que la mesure a corrigé dans WAC . . . 13

**6 Précision** **14**

6.1 En faveur de WAC . . . 14

6.2 En faveur des outils EZ . . . 15

**7 Formats de sortie** **16**

7.1 Ce que produit chaque outil . . . 16

7.2 Ce qui les distingue . . . 16

**8 Ce que la confrontation a appris à WAC** **18**

**9 Limites de la comparaison** **19**

CHAPITRE 1

# La démarche

## 1.1 Ce qui est comparé

Deux questions, deux comparaisons.

**Collecte : WAC face à KAPE**
Les deux outils collectent la même machine l'un après l'autre ; on compare ce que chacun copie, le temps qu'il y met, et l'empreinte SHA-1 des fichiers qu'ils copient tous deux.

**Traces laissées : WAC, KAPE et les outils EZ**
Chacun est lancé depuis le même état d'une même machine, sous Sysmon ; on relève ce qu'il y écrit, ce que Windows y écrit à cause de lui, et ce qu'un outil de surveillance en voit (chapitre 5).

**Décodage : WAC face aux outils EZ**
Sur les *mêmes pièces* — le magasin de pièces d'une collecte WAC —, WAC convertit un artefact et l'outil d'Eric Zimmerman correspondant le décode ; les deux sorties sont comparées enregistrement par enregistrement et champ par champ.

## 1.2 Qui a raison quand ils diffèrent

Un écart entre deux outils ne dit pas lequel se trompe. Chaque écart a donc été tranché par une troisième source, indépendante des deux :

- Windows lui-même : le XML qu'il rend pour un événement (`Get-WinEvent`), la date d'un fichier, l'identifiant d'objet NTFS (`fsutil objectid`), le registre en ligne, l'empreinte d'un fichier (`Get-FileHash`) ;
- une autre implémentation : `dissect.esedb` (Fox-IT) pour les bases ESE ;
- le format lui-même : numéros de séquence et sommes Marvin32 des journaux de ruche, lus octet par octet ;
- le recoupement de deux artefacts de la même machine : le Prefetch pour l'exécution, l'horodatage d'une ligne pour ses autres dates.

Chaque écart est ensuite classé : *défaut de WAC* (corrigé), *erreur de l'outil EZ* (vérifiée), ou *convention* de l'un des deux — deux formes d'une même valeur.

## 1.3 Le banc d'essai

| | |
|---|---|
| Machine | Windows 11 Professionnel (build 26200), machine virtuelle |
| Outils EZ | version 2026.5.0 (.NET 9.0.20) |
| KAPE | version 1.3.0.2, cible `KapeTriage` |
| WAC | version 2.0.0 et ses corrections issues de cette confrontation |
| Mesure des traces | clone de la machine, sans réseau, figé ; Sysmon 15.22 ; outils sur une clé USB (K :), une exécution par outil |

Tout se rejoue : `vmtest/ez-compare.sh <artefact>` chronomètre WAC et l'outil EZ sur les mêmes pièces puis compare leurs sorties ; `vmtest/kape-compare.sh` fait de même pour la collecte ; `vmtest/footprint/run-footprint.sh` rejoue la mesure des traces et `footprint-diff.py` l'analyse. Les outils EZ ont été lancés avec leur pleine précision de date (`--mp`, ou `--dt` au format à sept décimales quand `--mp` ne suffit pas), pour ne pas leur imputer une perte de précision qu'un réglage évite.

CHAPITRE 2

# Synthèse

| Artefact | Volume | Résultat | Erreurs de l'outil EZ, vérifiées |
|---|---|---|---|
| $MFT — MFTECmd | 362 081 enreg. | chemins, états, 8 dates identiques | tailles de 1 490 fichiers WOF et 3 journaux ouverts : Windows donne celles de WAC |
| EVTX — EvtxECmd | 190 613 évén. | champs système identiques ; valeurs expliquées | 6 valeurs fausses (Windows) ; 335 événements Perflib illisibles ; journaux vidés ignorés |
| Prefetch — PECmd | 379 fichiers | tout identique | empreinte privée de son zéro de tête (19) |
| Raccourcis — LECmd | 3 fichiers | identiques au 100 ns | — |
| Jump lists — JLECmd | 13 raccourcis | identiques au 100 ns | — |
| Shellbags — SBECmd | 27 dossiers | identiques | NodeSlot faux au premier niveau (7) ; HasExplored toujours faux |
| Amcache — AmcacheParser | 1 258 entrées | 124 colonnes identiques | 2 340 valeurs inventées ; SHA-1 partiel non signalé |
| ShimCache — AppCompat-CacheParser | 232 entrées | identiques | indicateur d'exécution trompeur |
| Corbeille — RBCmd | 19 fichiers | identiques | — |
| SRUM — SrumECmd | 60 804 lignes | identiques | 17 lignes d'énergie fausses (dissect) |
| Registre — RECmd | 9 artefacts | identiques | 4 ruches ramenées à un état ancien |
| Collecte — KAPE | 1 173 fichiers communs | SHA-1 identiques | — |

**À retenir**

Sur tout ce que les deux côtés lisent, WAC donne les mêmes valeurs que les outils de référence, ou des valeurs plus justes, établies à chaque fois par une source indépendante. La confrontation a aussi révélé 24 défauts de WAC, tous corrigés (chapitre 8), et des lacunes de couverture, ouvertes au suivi (section 6.2).

Lancé depuis une clé, WAC n'écrit rien sur la machine examinée : ni fichier, ni valeur de registre, ni événement — seules restent les traces que Windows laisse de tout programme lancé (Amcache, ShimCache). KAPE et les outils EZ, des programmes .NET, écrivent dans le registre de l'utilisateur et, pour les modules de KAPE, des fichiers sur le volume système (chapitre 5).

CHAPITRE 3

# Temps d’exécution

## 3.1 Décodage

Même machine, mêmes pièces ; chaque outil ne traite que son artefact.

| Artefact | WAC | Outil EZ | Ce que fait WAC en plus |
|---|---|---|---|
| $MFT (380 Mo) | 7,3 s | 8,1 s | deux volumes (C : et G :) contre un seul |
| EVTX (305 journaux) | 16,8 s | 22,0 s | message en clair reconstitué pour chaque événement |
| Registre (lot Kroll) | 2,0 s | 7,6 à 13,6 s | toutes les ruches rejouées, tous les artefacts du registre |
| SRUM (21 Mo) | 3,2 s | 2,5 s | 2 tables de plus, identités résolues |
| Prefetch (380) | 3,6 s | 0,7 s | heure locale datée, chemins avec lettre de volume |
| Raccourcis, jump lists | 1,8 s | 0,3 s | |
| Corbeille | 1,9 s | 0,5 s | |
| Amcache, ShimCache, shellbags | 2,0 s | 0,5 à 1,4 s | un seul passage pour les trois (clé registry) |

Sur les gros artefacts, WAC est plus rapide. Sur les petits, les outils EZ le sont : toute conversion WAC paie un coût fixe de 1,5 à 3,4 s, mesuré avec un artefact trivial. Ce coût est celui de la copie de travail vérifiée des ruches, du rejeu de leurs journaux, et de la lecture du fuseau et des profils du suspect. C’est ce qui permet à WAC de dater chaque heure locale avec le décalage en vigueur à sa date, et de donner la lettre de volume ; les outils EZ n’en donnent pas.

## 3.2 Collecte

| Outil, réglage | Durée | Pièces | Volume |
|---|---|---|---|
| KAPE, `KapeTriage` | 58 s | 1 047 | 0,97 Go |
| WAC, kit, sans binaires | 47 s | 1 898 | 2,54 Go |

| Outil, réglage | Durée | Pièces | Volume |
|---|---|---|---|
| WAC, kit, sans binaires, avant B-033 | 212 s | 2 622 | 1,69 Go |
| WAC, kit complet (`binary: unverified`), avant B-033 | 454 s | 61 294 | 3,6 Go |

Les deux premières lignes sont mesurées dans la même exécution, l'une après l'autre. À périmètre comparable, WAC collecte désormais plus vite que KAPE, en ramenant plus de deux fois plus de données. Il était cinq fois plus lent au départ (fiche B-033) :

— chaque lecture brute rouvrait son volume et reparcourait ses chemins depuis la racine ; les volumes sont maintenant ouverts une fois, avec le cache de leurs répertoires ;

— la collecte prenait les fichiers des 1 426 fournisseurs d'événements déclarés (739 Mo, 61 s) ; elle ne prend plus que ceux des fournisseurs que citent les journaux collectés (324 sur la VM, 178 Mo, 15 s), ce qui suffit à reconstituer le texte de chaque événement.

Le kit complet vérifie en outre la signature Authenticode de chaque exécutable de la machine et copie ceux dont la signature ne tient pas, ce que KAPE ne fait pas.

## 3.3 Collecte depuis un même état

Mesures de la comparaison des traces (chapitre 5) : chaque outil part du même état figé de la machine, lancé depuis la clé ; la durée est celle de la tâche qui le lance, entre parenthèses celle que l'outil annonce.

| Outil, réglage | Durée | Pris | Sur la clé |
|---|---|---|---|
| KAPE, `!SANS_Triage` | 127 s (115 s) | 2 291 fichiers | 2 286 fichiers, 1,25 Go |
| KAPE, `!SANS_Triage` et modules `!EZParser` | 235 s (223 s) | 2 288 fichiers, 14 outils | 2 345 fichiers, 1,63 Go |
| Outils EZ sur la machine vivante (6 outils) | 132 s | $MFT, journaux, ruches, Prefetch, Amcache, ShimCache | 36 CSV, 0,55 Go |
| WAC, collecte (réglage par défaut) | 592 s (588 s) | 11 830 pièces, 67 882 consignées | 8 240 fichiers, 4,9 Go |

La collecte de WAC est la plus longue parce qu'elle lit *chaque* exécutable de la machine : la conversion se fera ailleurs, et elle ne sait pas encore lesquels les artefacts citeront. Elle en calcule les empreintes, vérifie leur signature et ne copie que ceux dont la signature ne tient pas ; d'où 67 882 pièces consignées pour 11 830 copiées. La conversion, sur le poste

d’analyse, ne relit rien de la machine (chapitre 5). KAPE copie les fichiers de ses cibles sans les examiner.

CHAPITRE 4

# Ce que chacun prélève

## 4.1 Les mêmes pièces, des fichiers différents

Mêmes exécutions que la section précédente : la collecte de WAC, KAPE avec `!SANS_Triage`, comparés par le chemin d'origine de chaque pièce (manifeste de WAC, journal de copie de KAPE).

| Pièces | Nombre |
|---|---|
| Prises par les deux | 1 151, dont 1 011 au même SHA-1 |
| Prises par KAPE seul | 1 140 : profil de l'utilisateur (930, navigateurs et applications), profil NetworkService (91), `SleepStudy` (79), `LogFiles` (17), `WDI` (16) |
| Prises par WAC seul | 10 679 : exécutables à signature non valide (dont 5 300 copies laissées par le banc de test dans `C:\wactest`), catalogues de signature (1 205), journaux d'événements et fichiers de ressources de leurs fournisseurs, Defender (98) |

Les 140 pièces communes d'empreinte différente sont des fichiers que Windows réécrit en permanence — journaux d'événements (71), Prefetch (38), ruches (6), SRUM (3) : les deux exécutions partent du même état, mais Windows les a modifiés depuis le démarrage, différemment d'une fois à l'autre. Leur date de création est identique dans 1 146 cas sur 1 151. La couverture propre à KAPE est celle que suivent déjà les fiches B-016, B-031 et B-032.

## 4.2 Ce qui est consigné de chaque pièce

| | WAC (manifeste) | KAPE (journal de copie) |
|---|---|---|
| Empreintes | MD5, SHA-1, SHA-256 | SHA-1 |
| Dates du fichier | les 4 de `$STANDARD_INFORMATION` et les 4 de `$FILE_NAME`, au 100 ns, en UTC et en heure locale du suspect | 3 de `$STANDARD_INFORMATION` (création, modification, accès), au 100 ns, en UTC, sans le changement de l'enregistrement MFT |

| | **WAC (manifeste)** | **KAPE (journal de copie)** |
|---|---|---|
| Emplacement | numéro d'enregistrement MFT | chemin seul |
| Méthode | dite pour chaque pièce (lecture brute NTFS, observation vivante) | copie ordinaire ; lecture brute « différée » pour les fichiers verrouillés (181 sur 2 292) |
| Échec | code et cause dans le manifeste, pièce par pièce | console et journal |
| Intégrité | manifeste scellé (SHA-256), revérifié à chaque conversion | — |

**À retenir**

Les dates de `$FILE_NAME` sont celles qu'un intrus ne réécrit pas avec les fonctions ordinaires : un fichier antidaté (« timestomping ») garde dans `$FILE_NAME` sa vraie date de création. Le journal de KAPE ne les a pas ; le manifeste de WAC les donne pour chaque pièce, avec l'enregistrement MFT qui permet de la retrouver dans `mft.json`.

CHAPITRE 5

# Traces laissées sur la machine examinée

## 5.1 La mesure

Un clone de la machine de test, sans réseau, est figé avec Sysmon installé ; les outils sont sur une clé USB, lettre K :. Chaque exécution part de ce même état : le disque système et la clé sont des couches neuves sur les images figées. Le démarrage passé, chaque outil est lancé de la même façon — une tâche planifiée de l'utilisateur, avec ses droits élevés, comme un examinateur depuis une invite élevée.

**Les fichiers**
Le numéro USN du journal des modifications de C : est relevé juste avant et juste après l'outil : tout fichier créé, modifié, renommé ou supprimé entre les deux y figure, même créé puis supprimé pendant l'exécution. Les relevés sont écrits sur la clé, après la borne de fin.

**L'auteur**
Sysmon donne l'arbre des processus de l'outil et ce que chacun a écrit, fichiers et registre. Une écriture d'un autre processus qui nomme l'outil (Amcache, Prefetch, BAM) est une réaction de Windows à son lancement.

**Le témoin**
Une exécution qui ne lance rien (`ver`) mesure le lanceur et l'activité propre de Windows ; ce qu'elle montre aussi n'est pas imputé à l'outil.

**Le différé**
Le ShimCache n'est écrit qu'à l'arrêt : chaque exécution se termine par un arrêt propre, un redémarrage et sa lecture.

## 5.2 Ce que chacun laisse

| Outil | Fichiers écrits sur C : | Registre écrit | Réactions de Windows |
|---|---|---|---|
| WAC (collecte) | aucun | aucun | Amcache : une entrée pendant l'exécution ; ShimCache : une entrée à l'arrêt |

| Outil | Fichiers écrits sur C : | Registre écrit | Réactions de Windows |
|---|---|---|---|
| KAPE, cibles | aucun | 4 valeurs `ZoneMap` d'Internet Settings dans la ruche de l'utilisateur ; 169 ouvertures en création (magasins de certificats, stratégies) | Amcache et ShimCache pour `kape.exe` |
| KAPE, cibles et modules | 28 : 14 dans `ProgramData\Microsoft\Windows\BreadcrumbStore`, 14 dans le Temp de l'utilisateur (`system-commandline-generated-files`) | les mêmes 4 valeurs, écrites par 14 des 15 processus (136 écritures) ; environ 2 600 ouvertures en création | Amcache et ShimCache pour KAPE et 14 outils |
| Outils EZ, en direct | aucun | aucune valeur ; 54 ouvertures en création (stratégies, `Tcpip`, `Internet Settings`) | Amcache et ShimCache pour les 6 outils |

Aucun processus d'aucun outil n'a fait écrire d'événement dans les journaux de Windows : ceux de la fenêtre de mesure sont les ouvertures de session des services, Defender, la stratégie de groupe, communs au témoin ou sans lien avec l'outil. Tous lisent les volumes en brut, ce que Sysmon relève (`RawAccessRead`) et qu'un EDR peut signaler.

**Ouverture en création**
`RegCreateKeyEx` crée la clé si elle manque et l'ouvre sinon ; Sysmon ne distingue pas les deux cas. Ce sont les bibliothèques .NET qui ouvrent ainsi les magasins de certificats et les clés de stratégie. Une clé créée change de date ; une clé seulement ouverte, non.

**Les valeurs `ZoneMap`**
Écrites par l'initialisation de WinINet dans chaque processus .NET qui la charge, elles modifient la ruche `NTUSER.DAT` de l'utilisateur examiné, donc son journal et la date de la clé.

**Les fichiers .NET**
Le runtime laisse une « miette » par assemblage chargé dans `BreadcrumbStore`, et la bibliothèque System.CommandLine un fichier par outil dans le Temp de l'utilisateur. Lancés seuls, les outils EZ n'en ont pas laissé ; lancés par KAPE, si.

**Prefetch et BAM**
Aucun des 22 programmes lancés depuis la clé USB n'a reçu de fichier Prefetch ni de valeur BAM, quand `cmd.exe`, lancé du disque système par la même tâche, a sa valeur BAM. Un outil copié sur la machine et lancé de là en recevrait.

**Le prérequis .NET**
KAPE et les outils EZ demandent le runtime .NET 9, présent sur le clone. Sur une machine qui ne l'a pas, l'installer écrit des centaines de fichiers et de clés ; WAC est un exécutable autonome.

## 5.3 Ce que la mesure a corrigé dans WAC

WAC effaçait l'écran au démarrage par `system("cls")`, qui lance `cmd.exe` : un processus créé sur la machine examinée et le Prefetch de `CMD.EXE` mis à jour, à chaque collecte. L'écran est désormais effacé par l'API de la console, WAC n'importe plus aucune fonction capable de lancer un processus, et le script de compilation refuse une version qui en importerait une.

> **À retenir**
> Chaque trace de WAC est expliquée : il ne lance aucun processus, n'écrit aucun fichier sur la machine, aucune valeur de registre, aucun événement. Restent celles que Windows laisse de tout programme lancé — une entrée Amcache, une entrée ShimCache à l'arrêt —, inévitables et décrites dans le dossier d'innocuité, et la lecture brute des volumes, visible d'un EDR.

CHAPITRE 6

# Précision

## 6.1 En faveur de WAC

**Journaux de ruche rejoués dans l'ordre**
Les journaux `.LOG1` et `.LOG2` gardent, derrière les entrées récentes, les restes d'une génération précédente. RECmd les applique dans l'ordre du fichier, donc après les plus récentes. Il ramène ainsi SYSTEM, SECURITY, DEFAULT et une ruche de profil de service à un état plus ancien : dans SYSTEM, la séquence 0x3618 au lieu de 0x3620, et quatre dernières exécutions BAM affichées 13 minutes trop tôt. WAC trie les entrées par séquence et s'arrête à la rupture de chaîne. Les journaux eux-mêmes le vérifient, sommes Marvin32 comprises.

**Événements des journaux vidés**
Un journal vidé garde ses enregistrements : WAC les relit et les marque `EvtRecovered`, sans jamais les présenter comme vivants ; EvtxECmd ne les voit pas.

**Valeurs d'événements comme Windows les rend**
Hexadécimal `0x3e7`, GUID entre accolades, booléens `true`, binaire `69060000`, dates avec leur `Z`, marques U+200E conservées : les formes qu'affiche l'Observateur d'événements et que l'on recherche. EvtxECmd en change huit, et se trompe sur six valeurs, par exemple un booléen `False` que Windows rend `true`.

**Rien d'inventé**
Une valeur absente de la source n'est pas émise. AmcacheParser écrit 2 340 valeurs par défaut absentes de la ruche (`IsPeFile: False` pour un exécutable PE), SrumECmd 7 086 (0 ou $-1$).

**SHA-1 partiel de l'Amcache signalé**
Pour un fichier de plus de 30 Mio, Amcache ne hache que les 31 457 280 premiers octets (mesuré sur `mrt.exe`). WAC le dit (`Sha1CoversBytes`) ; sans cela, la comparaison avec le SHA-1 du fichier entier ferait conclure à tort à une modification.

**Pas de champ trompeur**
L'indicateur « exécuté » du ShimCache, que donne AppCompatCacheParser, dit « non » pour 105 des 117 binaires dont le Prefetch prouve l'exécution. WAC ne le relève pas : ce « non » ferait conclure qu'un programme n'a jamais tourné.

**Précision au 100 ns**
WAC donne chaque date à la précision de sa source. Par défaut, PECmd, LECmd et SrumECmd arrondissent à la seconde, et LECmd ignore `--mp` dans son CSV.

**Heure locale datée**
Chaque date est donnée en UTC et en heure locale du suspect, avec le décalage en vigueur

à cette date. Les outils EZ ne donnent que l'UTC, sans le signaler par un `Z`.

**Lectures d'autrui vérifiées**
Les « FirstInteracted » et « LastInteracted » de SBECmd, le « CreatedOn » des comptes chez RECmd sont des lectures de dates d'écriture de clés. WAC publie ces dates elles-mêmes : chacun en tire la lecture qu'il veut, sans en hériter une toute faite. SBECmd donne en outre un `NodeSlot` faux (0) aux dossiers de premier niveau.

**Erreurs de lecture ESE de SrumECmd**
17 lignes d'énergie sur 887 portent le contenu d'une autre ligne ou un identifiant absent de la table ; dissect.esedb donne raison à WAC.

**Collecte**
Chaque pièce de WAC porte MD5, SHA-1 et SHA-256, les huit dates NTFS de son original et son entrée MFT, dans un manifeste scellé. KAPE donne un SHA-1 et trois dates.

## 6.2 En faveur des outils EZ

**Couverture du registre**
Le lot Kroll de RECmd lit des clés que WAC ignore : programmes installés, exclusions de Defender, RecentDocs, MountPoints2, réseaux connus, état de la mise à jour des dates d'accès NTFS, heure d'arrêt, entre autres. Fiche B-031.

**Couverture de fichiers**
KapeTriage ne copie plus, face à WAC, que les navigateurs et la chronologie Windows, prévus (B-016, B-010), et les `desktop.ini`, écartés. Les raccourcis hors de Recent, la ruche du profil Default, `$Secure:$SDS`, `$Boot`, `$UsnJrnl:$Max`, `$TxfLog`, les traces WPP de Defender et les clés maîtresses DPAPI, qu'il était seul à prendre, sont collectés et convertis depuis la fiche B-032.

**Vitesse sur les petits artefacts et en collecte**
Voir le chapitre précédent.

**Interprétations prêtes à lire**
Les outils EZ découpent et nomment certains champs : exécutable, date et description d'une application SRUM, noms des dossiers connus, extension d'un fichier. WAC donne la valeur brute, telle que Windows la stocke.

> **Limite**
> Ce que WAC ne collecte pas, il ne peut pas le mettre en balance : la couverture de KAPE reste plus large sur les navigateurs et plusieurs clés de registre, jusqu'à ce que B-016, B-031 et B-032 soient faits.

CHAPITRE 7

# Formats de sortie

## 7.1 Ce que produit chaque outil

| Outil | Formats | Remarque |
|---|---|---|
| AmcacheParser | CSV | 8 fichiers, un par table |
| AppCompatCacheParser | CSV | |
| RBCmd | CSV | |
| SrumECmd | CSV | 7 fichiers |
| EvtxECmd | CSV, JSON, XML | |
| JLECmd | CSV, JSON, HTML | |
| LECmd | CSV, JSON, HTML, XML | |
| MFTECmd | CSV, JSON, bodyfile | |
| PECmd | CSV, JSON, HTML | |
| RECmd | CSV, JSON | plus un CSV par greffon |
| SBECmd | CSV, JSON | |
| KAPE | copie des fichiers, journal de copie CSV | |
| WAC | JSON pour chaque artefact | un fichier par artefact, plus le manifeste scellé |

## 7.2 Ce qui les distingue

Quatre outils sur onze ne produisent que du CSV. Chez les autres, le JSON est la transcription de la ligne CSV : une ligne de texte par enregistrement (JSON Lines), à plat.

**Types**
Chez les outils EZ, les nombres sont des chaînes : un compteur d'exécutions s'écrit `"RunCount":"458"`, un numéro d'enregistrement `"EventRecordId":"21572"`. WAC écrit des nombres, des booléens, des chaînes.

**Structures**
Chez les outils EZ, une liste devient une chaîne jointe par des virgules (les fichiers chargés d'un Prefetch, ses répertoires), et les données d'un événement sont un JSON échappé *dans* une chaîne (`Payload`), à décoder une seconde fois. WAC imbrique : les volumes d'un Prefetch

et leurs répertoires, les éléments d'un raccourci, les paires `Name`/`Value` d'un événement.

**Dates**
Chez les outils EZ, l'UTC n'est pas signalé (`2026-09-30 23:04:46`) et la précision dépend d'une option. WAC donne l'ISO 8601 complet, en UTC (…`Z`) et en heure locale avec son décalage (…`+02:00`).

**Absence de valeur**
Les outils EZ écrivent une valeur par défaut, ou une cellule vide. WAC n'émet pas le champ ; un artefact non collecté est déclaré tel (`NotRequested`, `NotCollected`), pour qu'une liste vide ne passe pas pour une absence de trace.

**Provenance**
Le manifeste de WAC relie chaque valeur à la pièce dont elle vient, avec ses empreintes et ses dates d'origine.

**Console**
Les outils EZ détaillent leur travail sur la console ; certains y écrivent le décodage lui-même quand aucun fichier n'est demandé. WAC n'y écrit que l'avancement : tout ce qu'il trouve est dans les fichiers.

> **À retenir**
> Pour un analyste, la différence se voit au premier traitement : un JSON de WAC se charge tel quel dans un outil d'analyse ou dans WAE, sans rien redécouper ; une sortie EZ se traite comme un tableur, dont il faut reconvertir les nombres, redécouper les listes et redécoder les données d'événement.

CHAPITRE 8

# Ce que la confrontation a appris à WAC

Chaque défaut ci-dessous produisait un JSON valide et faux, ou incomplet. Tous sont corrigés, avec un contrôle qui les aurait pris.

| Artefact | Défaut corrigé |
|---|---|
| $MFT | enregistrements d'extension périmés fusionnés avec la base |
| EVTX | flottants tronqués à 6 chiffres ; binaire en « 69 06 00 00 » ; entiers hexadécimaux et GUID au mauvais format ; ProcessingErrorData ignoré (21 événements sans données) ; événements des journaux vidés non lus |
| Raccourcis | URI collée au bloc qui la suit ; lecture décalée de 2 octets sans liste d'ID ; blocs ExtraData non lus (machine, suivi) ; taille de la cible non lue ; DROID nuls émis |
| Jump lists | nombre d'ouvertures et rang MRU non lus ; format de DestList version 1 non pris en charge ; nom d'application altéré |
| Shellbags | dossier perdu avec sa descendance ; date de la clé du dossier, rang et NodeSlot absents ; dossier « Accueil » sans nom |
| Amcache | 2 tables sur 14 et 4 à 6 valeurs lues : ni SHA-1, ni taille, ni pilote, ni périphérique |
| ShimCache | tabulations des applications empaquetées remplacées par des espaces |
| SRUM | dates FILETIME laissées en nombres ; texte UTF-16 lu comme de l'ANSI |
| Registre | BAM limité aux comptes ayant un profil (un compte supprimé perdait ses exécutions) ; durée de premier plan de UserAssist non lue |
| Lancement | `system("cls")` lançait `cmd.exe` sur la machine examinée (mesure des traces) |
| Harnais | un contrôle qui plantait interrompait tous les suivants sans échec visible |

CHAPITRE 9

# Limites de la comparaison

— Une seule machine, virtuelle, sous Windows 11 : certains artefacts y sont peu nombreux (3 raccourcis de Recent, 2 clés USB). Les formats sont les mêmes sur une machine réelle, mais les cas rares n'y sont pas tous représentés.

— Les deux collectes se suivent au lieu d'être simultanées : les fichiers écrits entre elles ($MFT, journaux, Prefetch) diffèrent légitimement.

— Les outils EZ ont été lancés avec leurs options de précision maximale ; par défaut, ils perdent davantage.

— La mesure des traces se fait sur un clone sans réseau, en une exécution par outil : une écriture de Windows que le témoin n'a pas montrée n'est pas imputée à l'outil sans que Sysmon l'attribue à l'un de ses processus ; elle reste listée comme non attribuée dans les résultats de `footprint-diff.py`.

— L'audit de création de processus (Security 4688) n'est pas actif sur la machine de test : sur une machine où il l'est, chaque outil y laisse un événement par processus — un pour WAC, quinze pour KAPE et ses modules.

— La comparaison porte sur les artefacts communs. Ce que seul l'un des outils lit n'a pas de référence ici : son exactitude repose sur les contrôles du dossier de validation.